Algorithms

Mihir Rao

Based on Algorithm Design by Kleinberg & Tardos

Contents

1 Introduction: Some Representative Problems **2**
1.1 Stable matching . . . 2
1.2 The Gale–Shapley algorithm . . . 3
1.3 Why the algorithm works . . . 4
1.4 Which stable matching does it find? . . . 5
1.5 Five representative problems . . . 6

Chapter 1: Introduction: Some Representative Problems

The chapter introduces three parts of algorithm design: formulate the problem, construct an algorithm, and prove that it is correct and efficient. Stable matching develops this process in detail. Five further problems show how a small change in a problem's rules can require a different algorithmic approach.

1.1 Stable matching

An assignment of applicants to employers should be *self-enforcing*. If an applicant and an employer both prefer one another to their assigned partners, they have a reason to abandon the assignment. Stable matching rules out every such pair.

1.1.1 The model

Use the book's simplified setting:

- There are n men, $M = \{m_1, \ldots, m_n\}$, and n women, $W = \{w_1, \ldots, w_n\}$.
- Each person ranks every person on the other side, with no ties.
- Each person can have at most one partner.

Definition 1.1 [Matching and stability]. A matching is a set $S \subseteq M \times W$ in which no person appears twice. It is perfect when everyone appears exactly once.

For a perfect matching containing (m, w) and (m', w'), an unmatched pair (m, w') is an instability when

$$m \text{ prefers } w' \text{ to } w, \qquad w' \text{ prefers } m \text{ to } m'.$$

A matching is stable if it is perfect and has no instability.

Both people must prefer the change. One person's dissatisfaction alone does not make a pair unstable.

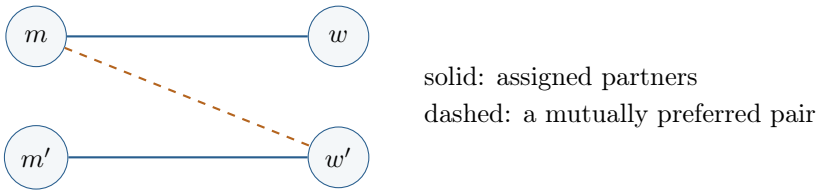


Figure 1: An instability in a perfect matching. Both m and w' would leave their partners to form the dashed pair.

1.1.2 The book's two small examples

Example 1: a unique stable matching. Let $M = \{m, m'\}$ and $W = \{w, w'\}$. Everyone agrees on the ordering of the other side:

man	preference	woman	preference
m	$w \succ w'$	w	$m \succ m'$
m'	$w \succ w'$	w'	$m \succ m'$

Here $a \succ b$ means that a is preferred to b. There are only two perfect matchings to check.

1. In $S_1 = \{(m, w), (m', w')\}$, m and w have their first choices. Neither would leave for the other person, so no unmatched pair can block S_1.
2. In $S_2 = \{(m, w'), (m', w)\}$, m and w both prefer each other to their assigned partners. The pair (m, w) blocks S_2.

Thus S_1 is the unique stable matching.

Example 2: two stable matchings. Now use these preferences:

man	preference	woman	preference
m	$w \succ w'$	w	$m' \succ m$
m'	$w' \succ w$	w'	$m \succ m'$

The same two perfect matchings are now both stable:

- $S_1 = \{(m, w), (m', w')\}$ gives both men their first choices. Every unmatched pair contains a man who prefers his current partner.
- $S_2 = \{(m, w'), (m', w)\}$ gives both women their first choices. Every unmatched pair contains a woman who prefers her current partner.

Stability therefore need not determine a unique answer. This example will also explain why the proposing side matters.

1.2 The Gale–Shapley algorithm

A proposal creates a tentative engagement. A woman keeps the best proposal she has received so far, but may later replace her partner. A free man proposes to his highest-ranked woman who has not already rejected him.

Algorithm 1.1: Gale–Shapley, with men proposing

```
Input: Complete, strict preference lists for M and W, with |M| = |W| = n
Mark everyone free
while some man m is free and has not proposed to every woman do
    Choose such a man m
    Let w be his highest-ranked woman not yet proposed to
    Record the proposal from m to w
    if w is free then
        Engage m and w
    else
        Let m′ be w's current partner
        if w prefers m to m′ then
            Engage m and w; make m′ free
        else
            Leave m free
return the set S of engaged pairs
```

For Example 1, one possible execution starts with m' rather than m:

Step	Proposal	Result
1	$m' \to w$	w accepts m'
2	$m \to w$	w keeps m; m' becomes free
3	$m' \to w'$	w' accepts m'

The first engagement was temporary. The final matching is S_1, as required.

1.3 Why the algorithm works

Two observations drive the proof:

- Once a woman receives a proposal, she remains engaged. Her partner can only improve in her preference order.
- A man's proposals move down his preference list. He never proposes to the same woman twice.

The first follows from the rule for accepting a new proposal; the second follows from always choosing the best untried woman.

Theorem 1.2 [Termination and correctness]. *Gale–Shapley terminates after at most n^2 proposals and returns a stable matching.*

Proof.

1. **Termination.** Each iteration adds a new proposed pair to a set $P(t) \subseteq M \times W$. Hence $|P(t)|$ increases by one and never exceeds n^2. The number of free people would not be a suitable progress measure, since replacing a partner may leave that number unchanged.

2. **Perfection.** A free man cannot have proposed to all n women. If he had, all n women would have received a proposal and would still be engaged. Their distinct partners would require n engaged men in addition to the free man, although only n men exist.

   Therefore, if a man were free at termination, he would have an untried woman and the loop could continue. Everyone must be matched when it stops.

3. **Stability.** Suppose the final pairs are (m, w) and (m', w'), but m and w' prefer each other. Since m prefers w' to w, he must have proposed to w' earlier and been rejected, either immediately or after an engagement. At that rejection, w' kept someone she preferred to m. Her partner could only improve afterward, so she prefers her final partner m' to m. This contradicts the proposed instability.

□

The bound counts proposals. An implementation also needs efficient ways to find a free man, advance his list, and compare two ranks; those details are developed in Chapter 2.

1.4 Which stable matching does it find?

The algorithm may choose any free man at each step. Nevertheless, all such executions give the same result when men propose.

Definition 1.3 [Valid partners]. A woman w is a valid partner of m if some stable matching contains (m, w). Let $\text{best}(m)$ be m's highest-ranked valid partner, and define

$$S^* = \{(m, \text{best}(m)) : m \in M\}.$$

A best valid partner is the best attainable *among stable matchings*. She need not be the first woman on the original preference list.

Theorem 1.4 [Proposer optimality]. *Every execution of men-proposing Gale–Shapley returns S^*.*

Proof. Assume that some execution rejects a man from a valid partner. Consider the first such rejection: w rejects m in favor of m', whom she prefers.

1. Because w is valid for m, there is a stable matching S' containing (m, w). Write (m', w') for the pair containing m' in S'; here $w' \neq w$.
2. The woman w' is a valid partner of m'. If m' preferred w' to w, he would have proposed to w' first. He could reach w only after being rejected by w', an earlier rejection by a valid partner. This would contradict the choice of the first rejection. Thus m' prefers w to w'.
3. In S', m' prefers w to his partner w', and w prefers m' to her partner m. The pair (m', w) blocks S', contradicting stability.

No man can therefore be rejected by a valid partner. Since he proposes in preference order and the final matching is stable, each man finishes with his best valid partner. □

This proves both that S^* is a matching and that proposal order does not change the result. In Example 2, men proposing produces S_1; women proposing produces S_2.

Proposition 1.5 [The receiving side]. *In S^*, every woman receives her worst valid partner.*

Proof. Suppose S^* contains (m, w), but another stable matching S' pairs w with m', whom she likes less than m. Let w' be m's partner in S'. Since $w = \text{best}(m)$ and w' is also valid, m prefers w to w'. Thus (m, w) blocks S', a contradiction. □

The guarantee concerns valid partners, not all partners on the preference list. Reversing the proposing side reverses these two guarantees.

1.5 Five representative problems

A graph $G = (V, E)$ records pairwise relationships: V is a set of nodes and an edge $\{u, v\} \in E$ joins two nodes. The five problems use related selection and assignment ideas, but have different structures.

1.5.1 Interval scheduling

Request i uses one resource from start time s_i to finish time f_i, where $s_i < f_i$. Two requests are compatible when

$$f_i \leq s_j \quad \text{or} \quad f_j \leq s_i.$$

A finish may coincide with the next start.

- **Goal.** Choose a compatible set A maximizing $|A|$.
- **Approach.** A suitable greedy rule processes requests in order and makes irrevocable choices. The rule and its proof come later.

1.5.2 Weighted interval scheduling

Each request now has a value $v_i > 0$. The compatibility condition stays the same, but the objective becomes

$$\max \left\{ \sum_{i \in A} v_i : A \text{ is compatible} \right\}.$$

For the book's example, suppose $v_1 > \sum_{i=2}^{n} v_i$. Every solution that excludes request 1 is worth less than request 1 alone. An optimum must therefore include request 1, regardless of its conflicts with other requests.

Values change which choices are sensible. Dynamic programming will compare alternatives through a table of optimal subproblem values. Setting every $v_i = 1$ recovers ordinary interval scheduling.

1.5.3 Bipartite matching

A graph is BIPARTITE if its nodes split into X and Y, and every edge joins the two sides. A matching is a set of edges with no shared ends; a perfect matching covers every node once.

- **Model.** Nodes in X may be jobs and nodes in Y machines. An edge means that the machine can perform the job.
- **Goal.** Find a matching of maximum size. If $|X| = |Y| = n$, a perfect matching exists exactly when that maximum is n.
- **Approach.** Augmentation enlarges a matching by revising some previous choices. It leads to the network-flow methods developed later.

This problem has allowed edges but no preference lists. Its objective is to match as many pairs as possible, rather than to prevent an instability.

1.5.4 Independent set

A node set $S \subseteq V$ is INDEPENDENT if no edge has both ends in S. The objective is to maximize $|S|$. An edge represents a conflict between two objects that cannot both be selected.

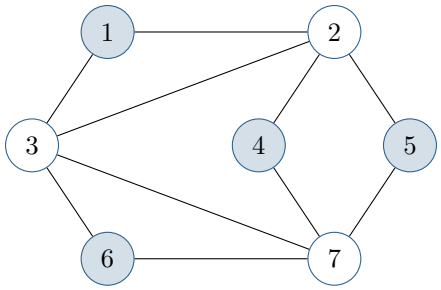


Figure 2: The book's independent-set example. The shaded nodes $\{1, 4, 5, 6\}$ have no edges between them.

The shaded set has size four. It is maximum: the triangle $\{1, 2, 3\}$ contributes at most one node, the edge $\{4, 7\}$ at most one, and the two remaining nodes $5, 6$ at most two.

Two earlier problems fit this model:

1. **Interval scheduling.** Make one node per interval and join intervals that overlap. Independent sets are exactly compatible schedules.
2. **Bipartite matching.** Given $G' = (V', E')$, make one node for *each edge* of G'. Join two new nodes when their original edges share an end. Independent sets in the new graph are exactly matchings in G'.

In the second construction, choosing nonadjacent new nodes means choosing original edges with no shared ends. This is why edges become nodes.

For general graphs, brute force examines all $2^{|V|}$ node subsets. The decision question, whether an independent set of size at least k exists, is NP-complete. A proposed set of size k is easy to verify: count its nodes and check that no pair is joined by an edge. This establishes a large enough independent set, not that the proposed set is maximum.

1.5.5 Competitive facility location

Two companies alternately open facilities in zones represented by a graph. Zone i has value b_i. Selecting a zone makes that node and its neighbors unavailable to both companies, so all selected zones together must form an independent set.

Player P_1 moves first. Given a target B, ask whether P_2 has a strategy that guarantees total value at least B, whatever P_1 does.

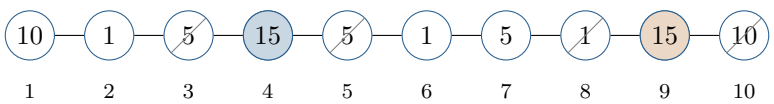


Figure 3: The book's zone values, with an illustrative legal sequence: P_1 selects zone 4 and P_2 selects zone 9. Node labels are values; small labels below are zone numbers. Crossed neighbors cannot be selected.

The displayed two moves are legal because zones 4 and 9 are not adjacent. They do not establish a winning strategy: P_1 could have chosen a different first zone. A strategy must specify responses to those different choices and to later moves.

This distinguishes the problem from verifying one independent set. A successful play is one sequence; a guaranteed outcome must handle opposing choices throughout the game. The chapter uses this distinction to introduce PSPACE-complete problems. These are believed to be harder than NP-complete problems, although that separation has not been proved.

Source. Chapter 1, Introduction: Some Representative Problems, *Algorithm Design*.